Introduction à DevOps

Vers une synergie entre Développement et Exploitation

Fondamentaux du DevOps

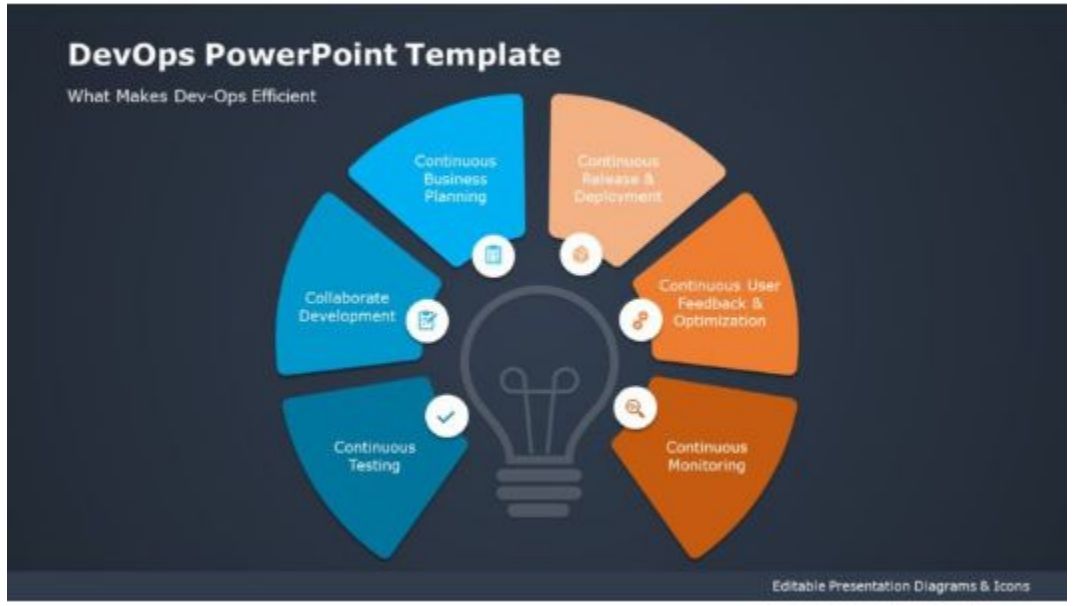
Définition et racines historiques

QU'EST-CE QUE LE DEVOPS ?

DevOps est une **culture** et un ensemble de pratiques qui favorisent la collaboration entre :

- → Les équipes de Développement (Dev)
- → Les équipes d'Exploitation (Ops)

> *Le but est de réduire les barrières afin de livrer des applications avec fiabilité.*



POURQUOI DEVOPS EST-IL NÉ ?

Le Silo

Séparation stricte provoquant des manques d'informations et des retards de production critiques.

Les Conflits

Le fameux "ça marche chez moi" vs "ça ne marche pas en prod". Une tension constante entre rapidité et stabilité.

Les Risques

Déploiements risqués, longs et manuels entraînant une baisse de qualité globale du produit.

EVOLUTION DES MÉTHODOLOGIES

Méthode	Approche Principale	Points Forts	Limites Principales
Waterfall	Séquentiel / Cascade	Structure claire et prévisible	Rigidité, retours trop tardifs
Agile	Itératif / Sprints	Flexibilité, feedback continu	Oublie souvent l'exploitation
DevOps	Cycle de vie complet	Automatisation, CI/CD, Fiabilité	Nécessite un changement culturel

Objectifs et Avantages

Les bénéfices tangibles pour l'organisation

COLLABORATION ET SYNERGIE

Objectifs Partagés

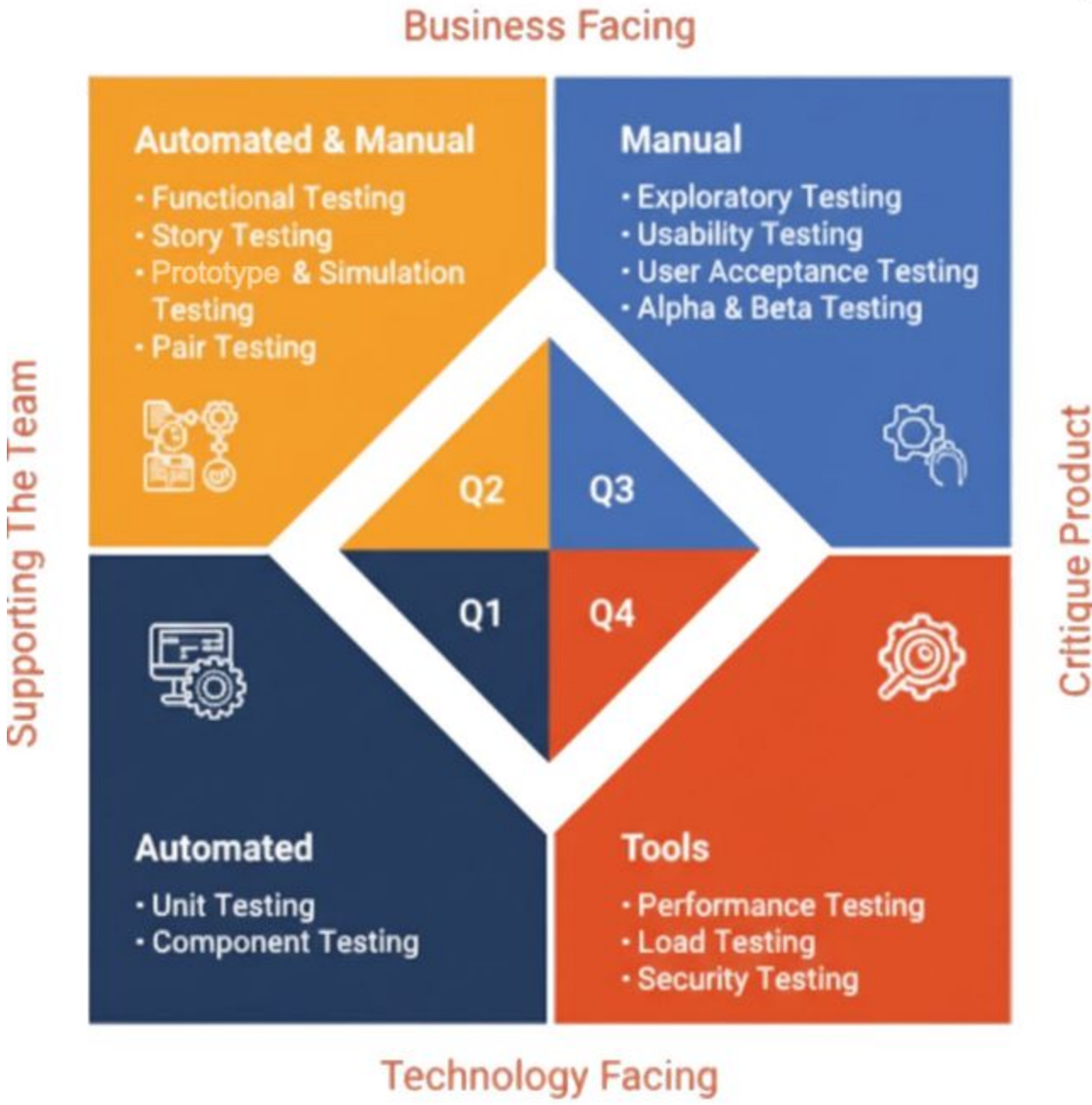
DevOps met en place une logique de coopération active en remplaçant les tensions par une communication fluide.

- → Responsabilités claires et partagées
- → Livraison d'un produit stable
- → Réduction drastique des silos

ACCÉLÉRATION DU TIME-TO-MARKET

Livrer des fonctionnalités plus fréquemment et corriger les bugs plus vite.

- → Automatisation des tests
- → Pipelines CI/CD robustes
- → Standardisation des environnements



Agile Testing Quadrants

L'AMÉLIORATION CONTINUE

Mesurer

Collecter des métriques sur ce qui fonctionne réellement en production.

Analyser

Identifier les points faibles, les bugs récurrents et les goulots d'étranglement.

Améliorer

Corriger, automatiser davantage et recommencer le cycle pour une qualité optimale.

QUESTIONS ?

Principes de Base de DevOps

Culture, Valeurs et Cycle de Vie

La Culture DevOps

Plus qu'un outil, un état d'esprit

VALEURS FONDAMENTALES (1/2)

Collaboration

Dev et Ops travaillent ensemble dès le début du projet, au lieu de se passer le relais à la fin.

Communication

Les informations circulent de manière claire et rapide, essentielle pour la résolution d'incidents.

Responsabilité

La qualité, la performance et la disponibilité concernent l'ensemble des équipes sans distinction.

VALEURS FONDAMENTALES (2/2)

Automatisation

Tout processus répétitif doit être automatisé pour réduire drastiquement les erreurs humaines.

Feedback Rapide

Les retours (tests, monitoring) doivent arriver vite pour permettre une correction précoce.

Amélioration

Apprendre des erreurs passées pour améliorer le système et les pratiques en permanence.

LE MODÈLE CALMS

5

Piliers DevOps

Structurer l'esprit DevOps

Culture : L'aspect humain avant tout.

Automation : L'outil au service de l'efficacité.

Lean : Éliminer les gaspillages.

Measurement : Mesurer pour s'améliorer.

Sharing : Partager les succès et les échecs.

CHANGEMENT DE POSTURE

Modèle Classique

Dev développe → Ops déploie

Conséquences : Tensions, retards, déploiements risqués et manque de visibilité globale.

Modèle DevOps

Dev + Ops collaborent

Conséquences : Conception commune, automatisation partagée et objectifs de stabilité unifiés.

IMPACT SUR LES PERFORMANCES

- **Fiabilité :** Réduction significative des erreurs lors des déploiements.
- **Rapidité :** Réduction drastique du temps de mise en production (Time-to-market).
- **Stabilité :** Meilleure résilience des applications en production.
- **Satisfaction :** Amélioration de l'expérience utilisateur final.
- **Sérénité :** Livraison logicielle moins stressante et mieux contrôlée.

Cycle de Vie DevOps

De la planification au feedback continu

PHASES DU CYCLE (1-4)

Plan
Définition des besoins et priorisation.

Code
Écriture et gestion de versions (Git).

Build
Compilation et intégration (CI).

Test
Tests automatisés et détection d'erreurs.

PHASES DU CYCLE (5-8)

Deploy
Mise en production automatisée (CD).

Operate
Gestion de l'infrastructure et serveurs.

Monitor
Surveillance des ressources et logs.

Feedback
Analyse et amélioration continue.

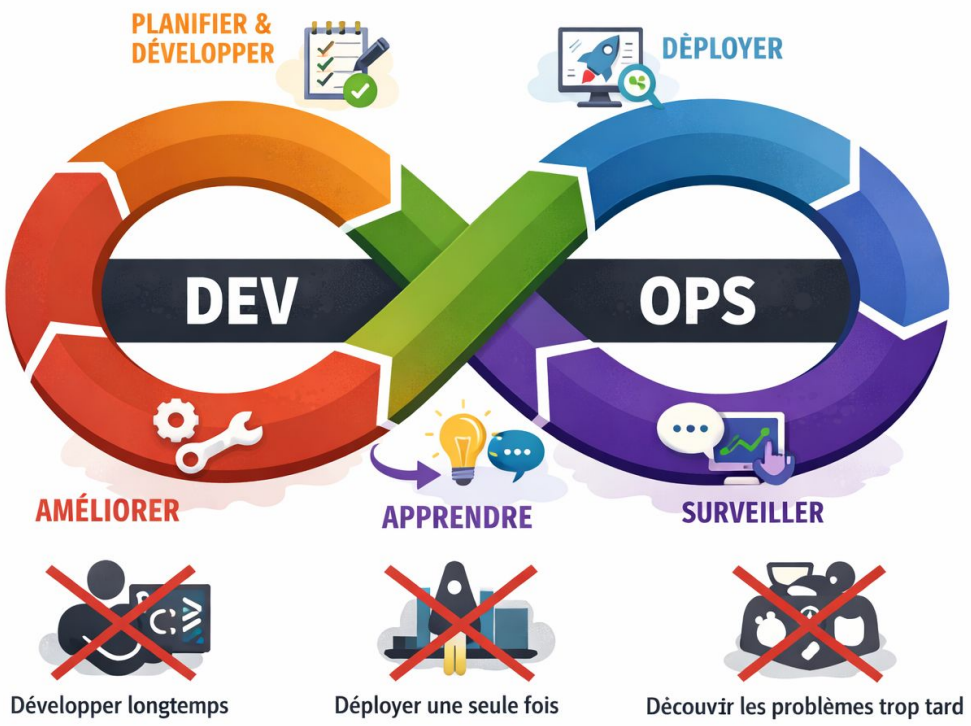
UNE BOUCLE CONTINUE

L'infini DevOps

Le point central est que ces étapes forment une **boucle sans fin**.

On développe, déploie, surveille, apprend, puis on améliore.

Ce cycle itératif permet d'éviter les découvertes tardives de problèmes et garantit un produit toujours plus robuste et transparent.



Questions ?

Échangeons sur les principes fondamentaux de DevOps

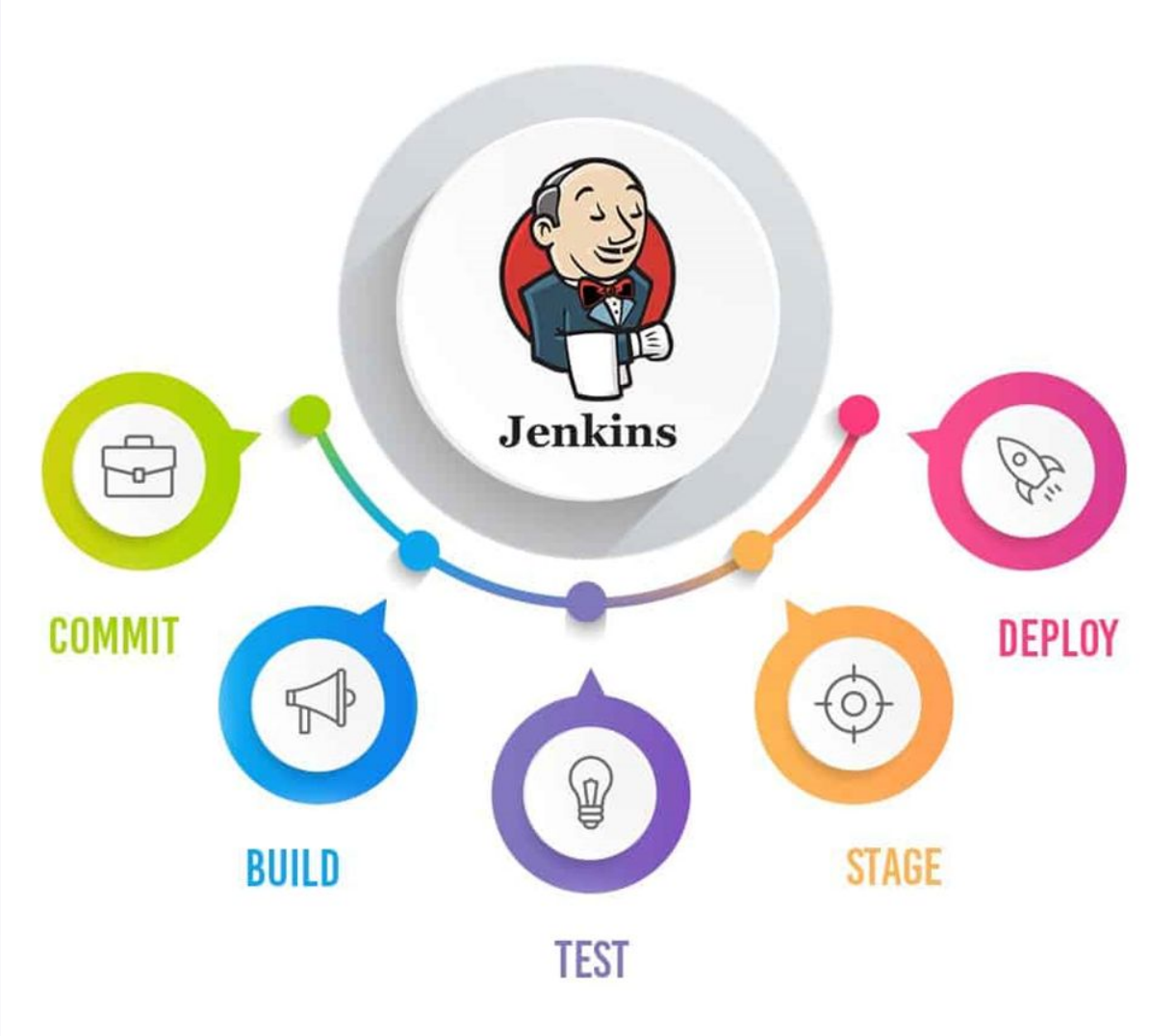
Initiation à la démarche DevOps

Outils DevOps

Jenkins & Docker : Automatisation et Conteneurisation

Jenkins pour CI/CD

Le chef d'orchestre de l'intégration continue

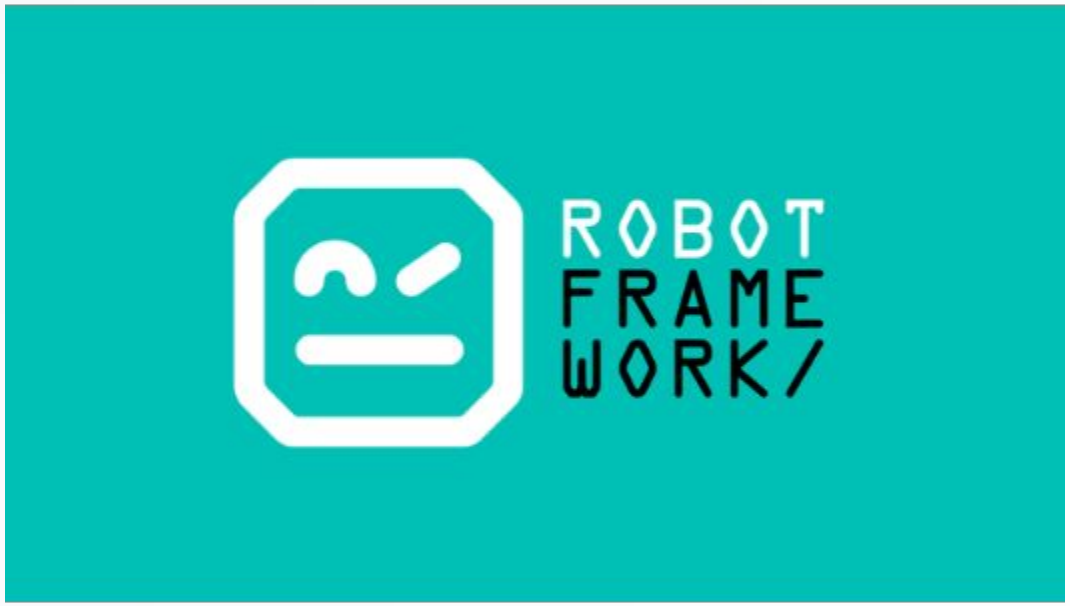


Pourquoi automatiser avec Jenkins ?

Les processus manuels classiques ralentissent le développement :

- 🚫 Compilation locale isolée
- ⚠️ Tests irréguliers ou absents
- 🕒 Livraisons lentes et risquées

> **La CI/CD** apporte une réponse systématique et fiable à ces goulots d'étranglement.



CI vs CD : Concepts Clés

Intégration Continue (CI)

Fusion fréquente du code dans un dépôt Git. Déclenche automatiquement compilation, tests unitaires et validation qualité.

Objectif : Détecter les erreurs au plus tôt.

Livraison Continue (CD)

Automatisation de la préparation au déploiement. Création d'artefacts (.jar, images Docker) prêts à la mise en production.

Objectif : Rendre le déploiement reproductible.

Le Rôle de Jenkins

Exécuter

Lancement automatique des tâches de build et de test à chaque commit.

Standardiser

Garantie que chaque version passe par les mêmes étapes de validation.

Tracer

Fourniture d'un historique complet et de rapports sur la qualité du code.

Cycle d'un Pipeline CI/CD

Checkout

Récupération Git

Build

Compilation

Test

Validation Auto

Package

Création Artefact

Deploy

Mise en Prod

Jenkins orchestre cette succession d'étapes (stages).

Bonnes Pratiques Jenkins

- **Pipeline as Code :** Définir le pipeline dans un fichier Jenkinsfile versionné dans Git.
- **Tests Systématiques :** Une pipeline sans tests n'est qu'un "déploiement automatique de bugs".
- **Gestion des Secrets :** Ne jamais écrire de mots de passe ou clés en clair dans le code du pipeline.
- **Feedback Rapide :** Fournir immédiatement les logs et rapports de succès ou d'échec.

Docker & Conteneurs

Garantir la portabilité des environnements

Fin du "Ça marche chez moi"

Les différences entre environnements (versions d'OS, dépendances, config) créent des bugs imprévisibles.

Docker résout cela en emballant :

- Le code applicatif
- Les bibliothèques et dépendances
- La configuration système

Conteneur vs Machine Virtuelle

Critère	Machine Virtuelle (VM)	Conteneur Docker
Système	OS Complet (Guest OS)	Partage le noyau Host
Poids	Plusieurs Go	Quelques Mo
Vitesse	Minutes pour démarrer	Millisecondes
Échelle	Faible densité	Haute densité (Scale out)

Concepts Fondamentaux

Dockerfile

La recette décrivant comment construire l'image (OS, dépendances, commande).

Image

Un modèle immuable et statique de votre application prête à l'emploi.

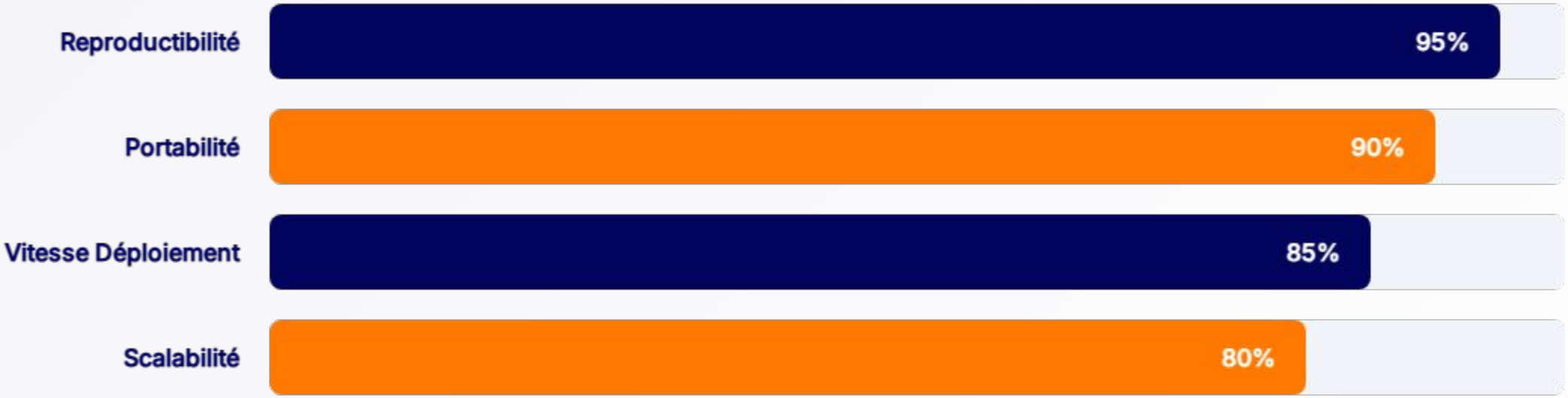
Conteneur

L'instance active et en exécution d'une image Docker.

Registry

Lieu de stockage des images (Docker Hub, Registry privé).

Avantages Clés de Docker

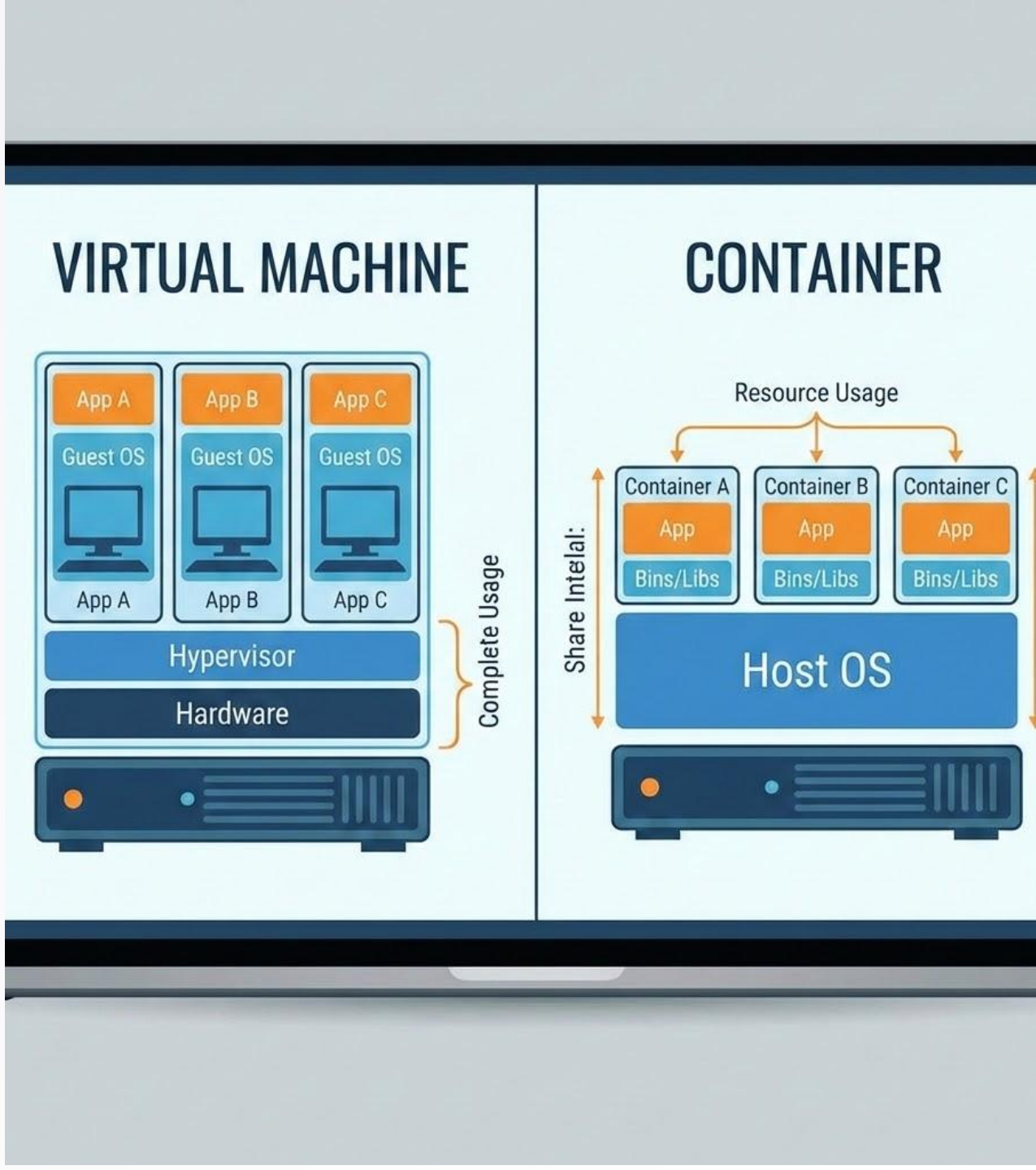


Indice d'impact sur l'efficacité opérationnelle des équipes DevOps.

Docker dans la Pipeline

Docker devient le format standard de livraison logicielle :

1. Jenkins compile et teste.
2. Jenkins construit l'image Docker.
3. L'image est poussée vers un Registry.
4. Déclenchement du déploiement.



Questions ?

Merci pour votre attention sur les outils DevOps.

Jenkins & Docker : Le socle de l'automatisation moderne

KUBERNETES & ANSIBLE

Orchestration de Conteneurs et Gestion de Configurations

AUTOMATISATION À L'ÉCHELLE INDUSTRIELLE

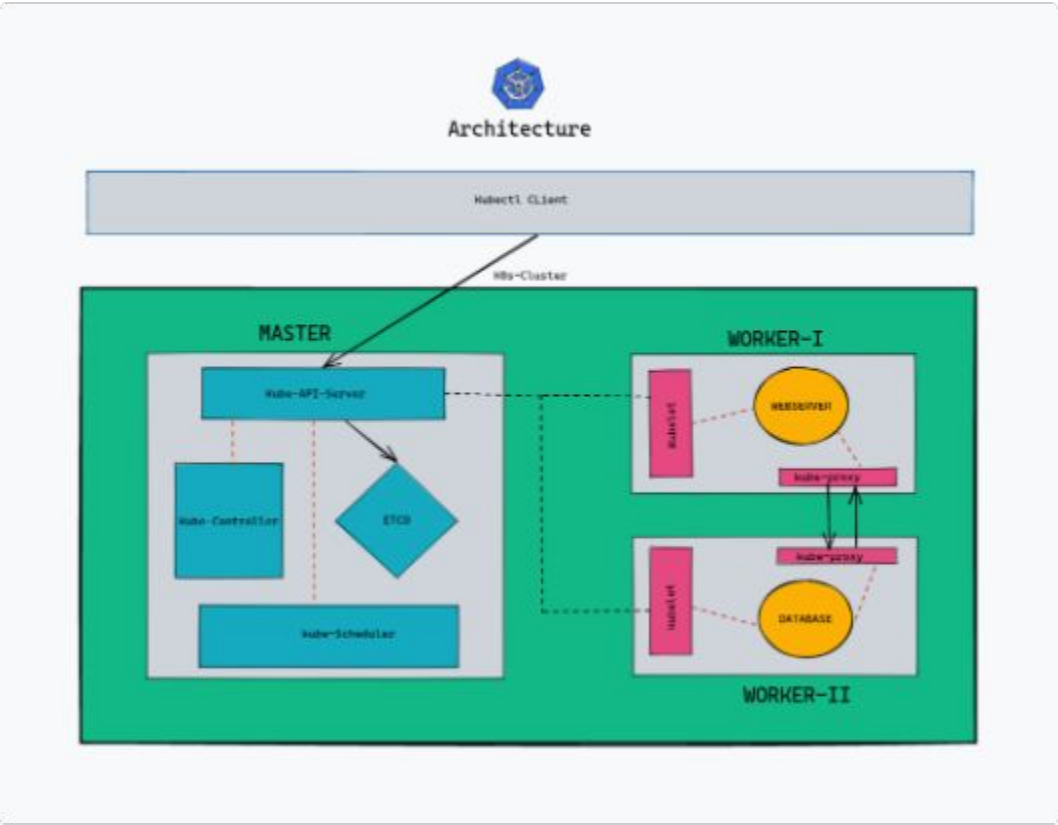
Kubernetes

Le standard de l'orchestration moderne

Pourquoi Kubernetes ?

Si Docker lance les conteneurs, Kubernetes les **gouverne** à grande échelle :

- **Auto-réparation :** Redémarrage auto en cas de crash.
- **Répartition :** Distribution sur plusieurs machines.
- **Scaling :** Gestion de la montée en charge.
- **Rolling Updates :** Mises à jour sans interruption.



Concepts Clés : Infrastructure

Cluster

Ensemble de machines (physiques ou virtuelles) gérées comme une seule entité par Kubernetes.

Node

Une machine individuelle au sein du cluster, chargée d'exécuter les applications.

Pod

L'unité la plus petite : contient un ou plusieurs conteneurs partageant des ressources.

Concepts Clés : Gestion

Deployment

Définit l'état souhaité : nombre de réplicas, stratégie de mise à jour et auto-remplacement.

Service

Expose un ensemble de pods via une adresse IP stable et gère le load-balancing.

kubectl

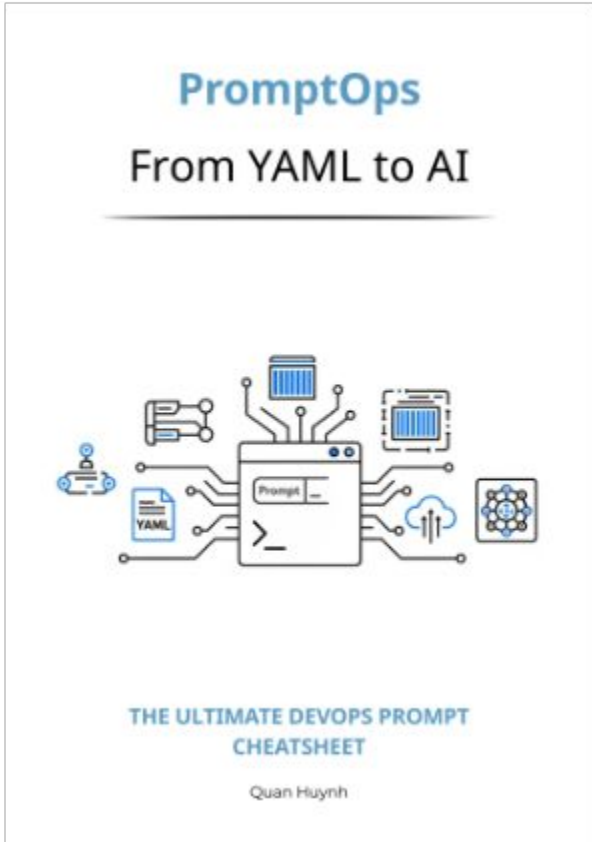
L'outil de ligne de commande universel pour interagir et piloter le cluster Kubernetes.

Déploiement d’une Application

Le déploiement repose sur une approche **déclarative** via des manifestes YAML :

- Définition de l'image Docker source.
- Nombre de réplicas souhaités.
- Configuration de l'exposition réseau.

> *K8s se charge de maintenir cet état cible en permanence.*



Pourquoi Kubernetes est un Outil DevOps ?

100%

Automatisation

- **Standardisation :** Même processus partout.
- **Résilience :** Haute disponibilité native.
- **Agilité :** Déploiements et retours arrière rapides.
- **Modernité :** Prêt pour le Cloud Native.

Ansible

Gestion de configurations et Infrastructure as Code

Le Problème du Manuel

Configuration Artisanale

Commandes "copier-coller", manque de documentation et aucune traçabilité des modifications.

Résultat : Environnements incohérents.

Serveurs "Magiques"

Dépendance vis-à-vis d'une seule personne experte. Risques élevés d'erreurs humaines lors des mises à jour.

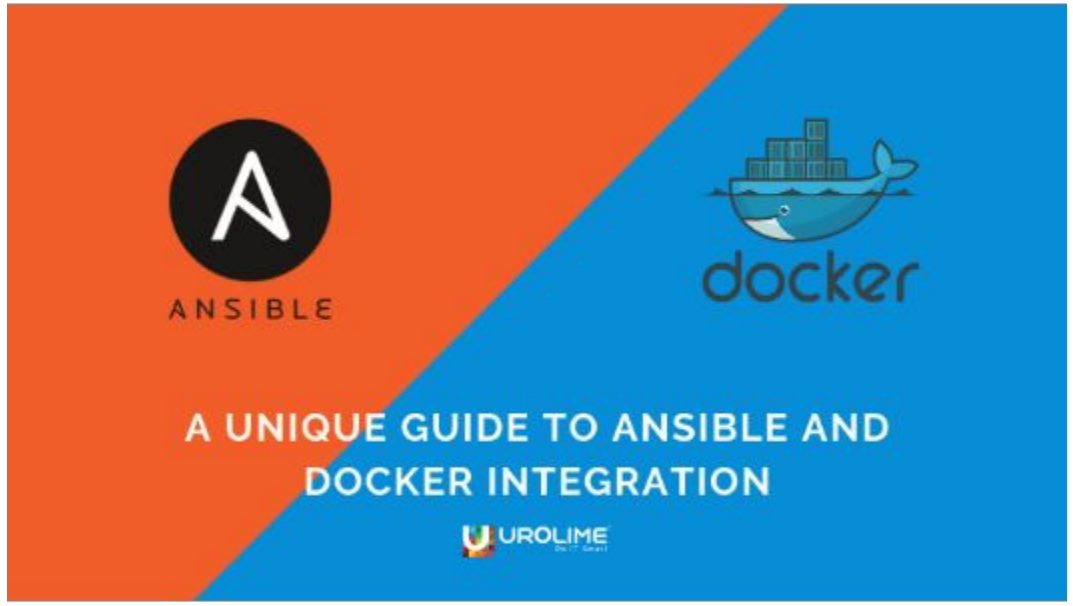
Résultat : Fragilité opérationnelle.

Ansible : L'Automatisation YAML

Ansible permet de piloter votre parc informatique avec simplicité :

- Configuration des serveurs.
- Installation de logiciels.
- Déploiement d'applications.

Sans agent : Ansible se connecte simplement via SSH.



Concepts Fondamentaux Ansible

Inventaire

La liste structurée des machines (groupes, adresses) à piloter.

Modules

Actions prêtes à l'emploi : apt, yum, copy, etc.

Playbook

La suite d'instructions YAML décrivant l'état final désiré.

Idempotence

Produit le même résultat peu importe le nombre d'exécutions.

Ansible dans la Démarche DevOps

0

Écart de Config

- **Exploitation Auto :** Libère du temps pour l'innovation.
- **Reproductibilité :** Recréation d'infra en quelques minutes.
- **Pipeline Intégré :** Déclenchement via Jenkins.
- **Traçabilité :** Tout est versionné dans Git.

Questions ?

Kubernetes & Ansible : Le duo de choc de l'automatisation

Vers une infrastructure agile et résiliente

L’Automatisation : Pilier DevOps

Réduire les risques et fluidifier la livraison logicielle

VALEUR • AGILITÉ • FIABILITÉ

Pourquoi Automatisons-nous ?

La collaboration nécessite une base technique robuste

LES RISQUES DES PROCESSUS MANUELS

Le manuel multiplie les "zones de risque" :

- Étapes oubliées ou ordre inversé.
- Incohérence des versions installées.
- Dépendances manquantes entre serveurs.

> *Plus on déploie souvent, plus ces risques se multiplient de manière exponentielle.*

DÉFIS DE L'ÉCHELLE ET DE L'HUMAIN

Incohérence

Le serveur de test diffère de la production, créant des bugs "invisibles" jusqu'au déploiement final.

Dépendance

Une seule personne connaît la procédure "magique". Son absence bloque tout le cycle de livraison.

Traçabilité

En cas de panne, impossible de savoir qui a modifié quoi, quand, et surtout pourquoi.

Scripts DevOps : Bash et Python

La première étape vers l'autonomie technique

BASH VS PYTHON : LEQUEL CHOISIR ?

Bash (Scripts Shell)

Idéal pour enchaîner des commandes Linux, gérer des fichiers système et manipuler des services.

Focus : Administration système rapide et légère.

Python

Privilégié pour la manipulation de données complexes, les interactions API et la maintenabilité.

Focus : Automatisation avancée et rapports structurés.

BONNES PRATIQUES DE SCRIPTING

- **Produire des logs :** Expliquer chaque action pour faciliter le débogage.
- **Gérer les erreurs :** Arrêter le script immédiatement si une étape critique échoue.
- **Zéro valeurs en dur :** Utiliser des variables d'environnement ou fichiers de configuration.
- **Sécurité des secrets :** Ne jamais stocker de mots de passe ou clés en clair dans le code.

Infrastructure as Code (IaC)

Traiter l'infrastructure comme du logiciel

TERRAFORM : LOGIQUE DÉCLARATIVE

1

État Final Souhaité

Décrire le "Quoi", pas le "Comment"

Contrairement aux scripts qui listent des commandes, Terraform décrit la cible.

Script : "Crée une machine, puis ouvre un port..."

IaC : "Je veux une machine configurée ainsi avec ces accès."

CONCEPTS CLÉS DE TERRAFORM

Provider

Connecteur vers les plateformes (AWS, GCP, Azure, VMware) pour piloter leurs APIs.

State

Mémoire de l'infrastructure réelle pour comparer et appliquer les changements proprement.

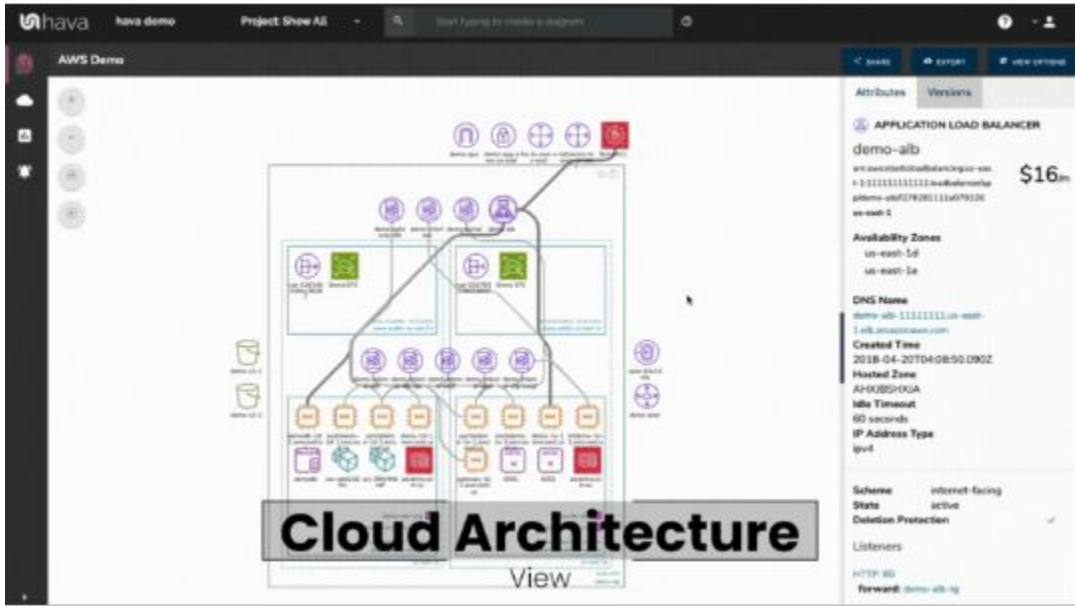
Plan & Apply

Simuler l'impact (Plan) avant de l'appliquer réellement (Apply) pour réduire les erreurs.

LES BÉNÉFICES DE TERRAFORM

Terraform transforme l'infrastructure en un objet versionné et traçable :

- Cloner des environnements en minutes.
- Standardisation Dev / Test / Prod.
- Collaboration en équipe via Git.



Questions ?

L'automatisation au service de l'excellence opérationnelle

Merci de votre attention !

Kubernetes & Ansible

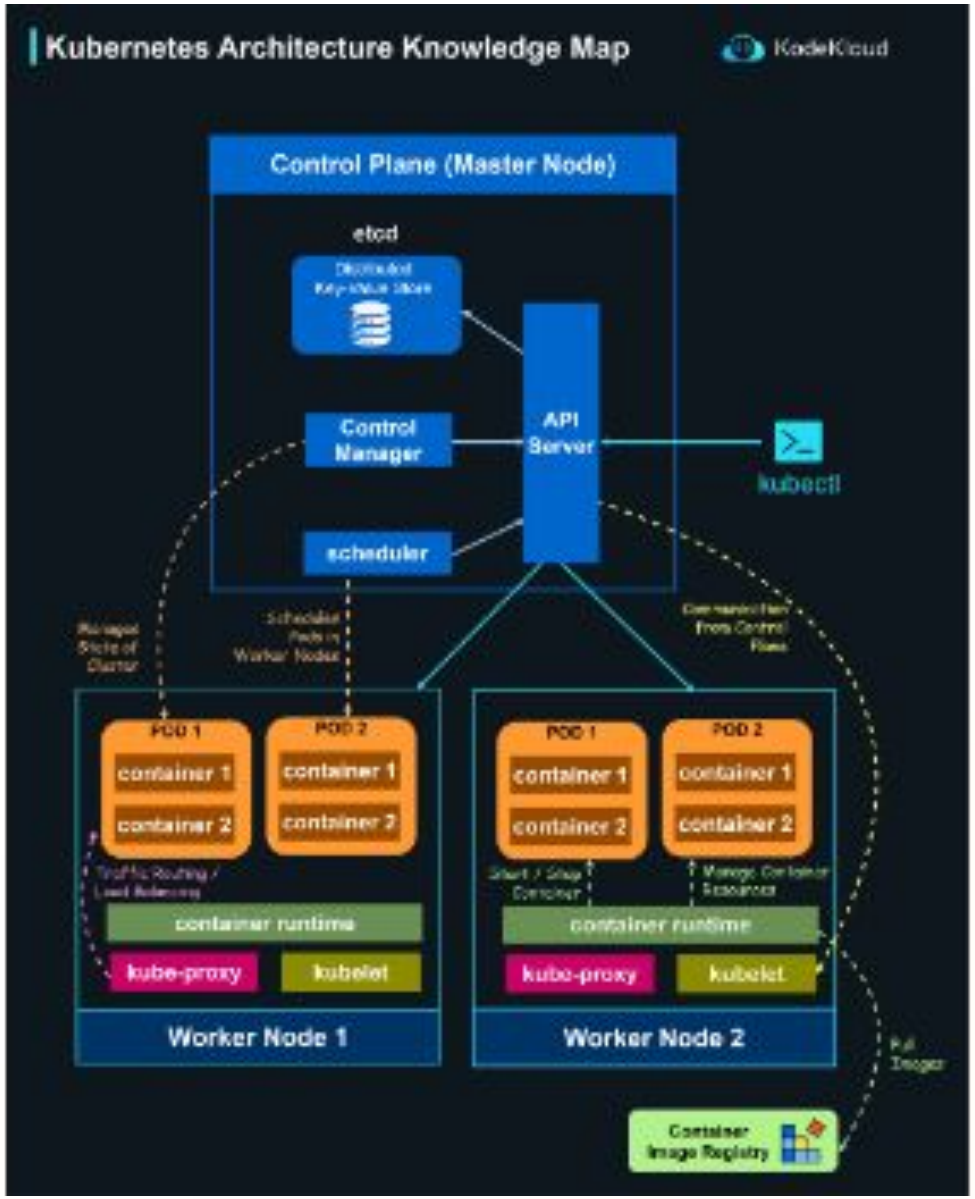
Automatisation, Résilience et Vision Globale DevOps

Orchestration avec Kubernetes

POURQUOI LE K8S AVANCÉ ?

Pour une application en production, Docker seul ne suffit pas. On exige :

- Une disponibilité continue 24/7
- Des mises à jour sans interruption
- Une adaptation à la charge variable
- Une résilience face aux pannes



L'ORCHESTRATION EN ACTION

Stabilité

Redémarre automatiquement les conteneurs tombés pour maintenir l'état souhaité.

Répartition

Équilibre la charge intelligemment sur l'ensemble du cluster de machines.

Fiabilité

Automatise la surveillance et la mise à jour progressive des applications.

MISES À JOUR & ROLLBACK

- **Rolling Update :** Déploiement progressif de nouveaux pods tout en retirant les anciens pour éviter toute coupure de service.
- **Rollback Rapide :** Capacité de revenir instantanément à une version précédente stable si une mise à jour échoue.
- **Sécurité de Livraison :** On ne livre souvent que si l'on peut revenir en arrière rapidement et sans risque.

LIVENESS & READINESS PROBES

Readiness Probe

Vérifie si l'application est prête à recevoir du trafic utilisateur réel.

Liveness Probe

Détecte si le conteneur est vivant ou bloqué, déclenchant un redémarrage si besoin.

Robustesse

Ces sondes garantissent que seuls les pods fonctionnels servent les requêtes.

RÉSILIENCE & SCALING

99.9

Disponibilité Cible

Gestion Intelligente

Kubernetes surveille l'état réel et le compare à l'état souhaité. Si un pod ou une machine tombe, il replanifie ailleurs instantanément.

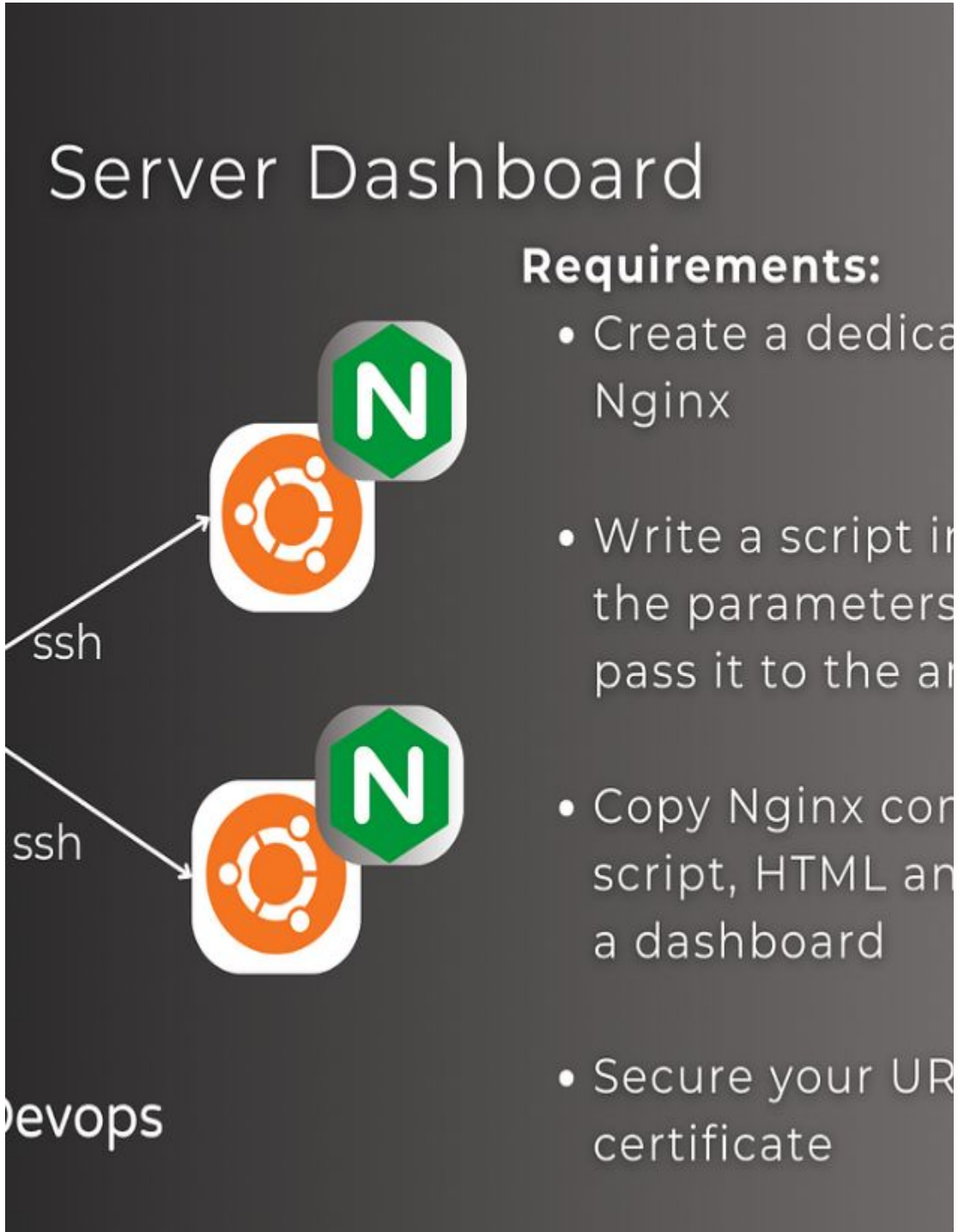
Le **Autoscaling** ajuste dynamiquement le nombre de réplicas selon la consommation de CPU ou mémoire.

Ansible pour le Déploiement

LE DÉPLOIEMENT RÉEL

Déployer n'est pas "copier le code". C'est un processus complexe :

- Gestion des dépendances
- Variables d'environnement
- Configuration des ports
- Redémarrages contrôlés



STRUCTURER ANSIBLE

Rôles

Découpe le déploiement en blocs logiques et réutilisables.

Templates

Génère dynamiquement les fichiers de configuration via Jinja2.

Handlers

Redémarre les services uniquement quand une modification a eu lieu.

LA CHAÎNE DEVOPS GLOBALE

Jenkins

Build & Test

Docker

Image stable

Kubernetes

Orchestration

Ansible

Déploiement

Synthèse : Automatisation

Chaque outil joue un rôle crucial dans la stabilité

Questions ?

Échangeons sur Kubernetes et Ansible

Merci pour votre attention !

LA SÉCURITÉ AU CŒUR DU DEVOPS

Stratégies, Outils et Culture DevSecOps

ORDRE DU JOUR

- L'enjeu central de la sécurité
- Définition et philosophie DevSecOps
- Risques dans un environnement DevOps
- L'arsenal des outils (SAST, DAST)
- Intégration Jenkins & CI/CD
- Sécurité spécifique à Kubernetes
- Gestion des Secrets et Chiffrement
- Synthèse et Équilibre Agilité/Sûreté

I. L'Évolution du Besoin Sécuritaire

Pourquoi la sécurité n'est plus une option de fin de projet

LE MODÈLE TRADITIONNEL "SILOTÉ"

Auparavant, la sécurité était une étape finale, isolée du flux de développement.

- Développement → Test → Déploiement
- Vérification de sécurité **à la fin**
- La sécurité devient un goulot d'étranglement

LE PROBLÈME : ARRIVER TROP TARD

Retours Arrière

Modifier le code ou l'architecture à la dernière minute est coûteux et frustrant.

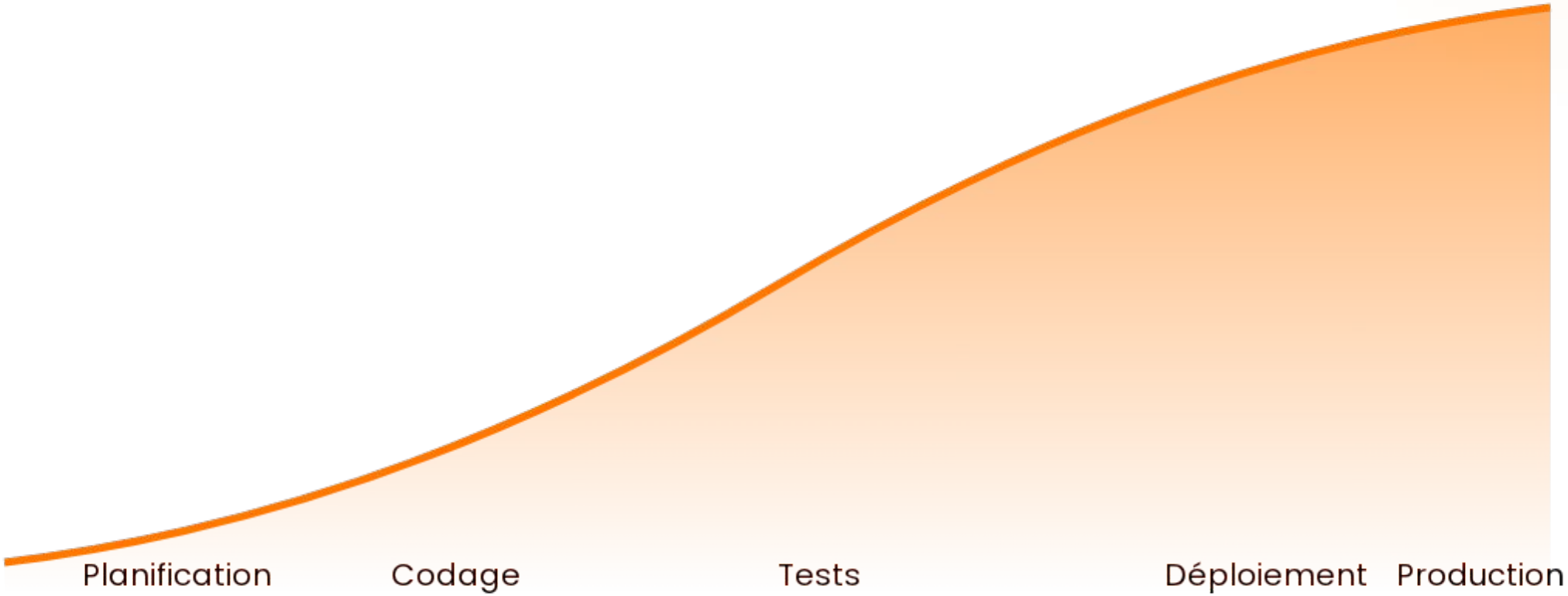
Risque Critique

Découvrir une faille en production expose directement les données utilisateurs.

Coût Élevé

Plus une faille est détectée tard, plus sa correction coûte cher à l'organisation.

COÛT DE CORRECTION DES FAILLES

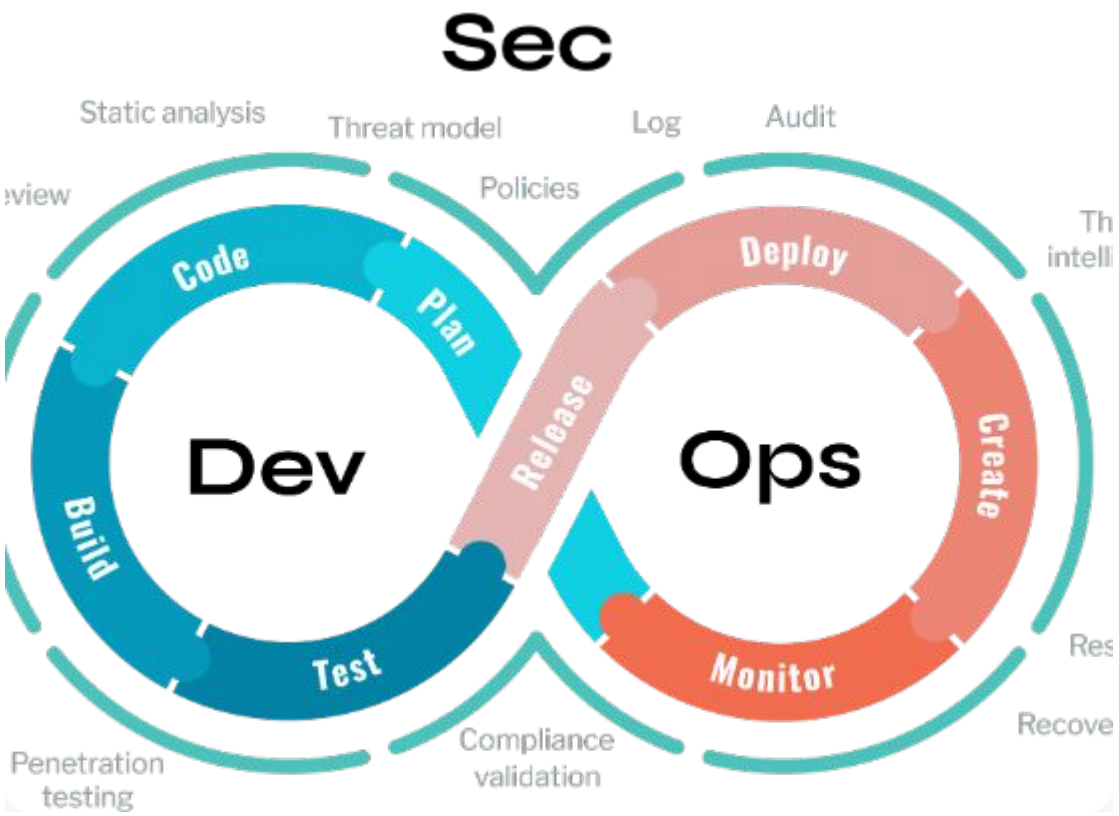


Représentation conceptuelle de l'augmentation exponentielle des coûts selon la phase de détection.

II. La Philosophie DevSecOps

Intégrer la sécurité dans l'ADN du pipeline

QU'EST-CE QUE LE DEVSECOPS ?



Dev + Sec + Ops

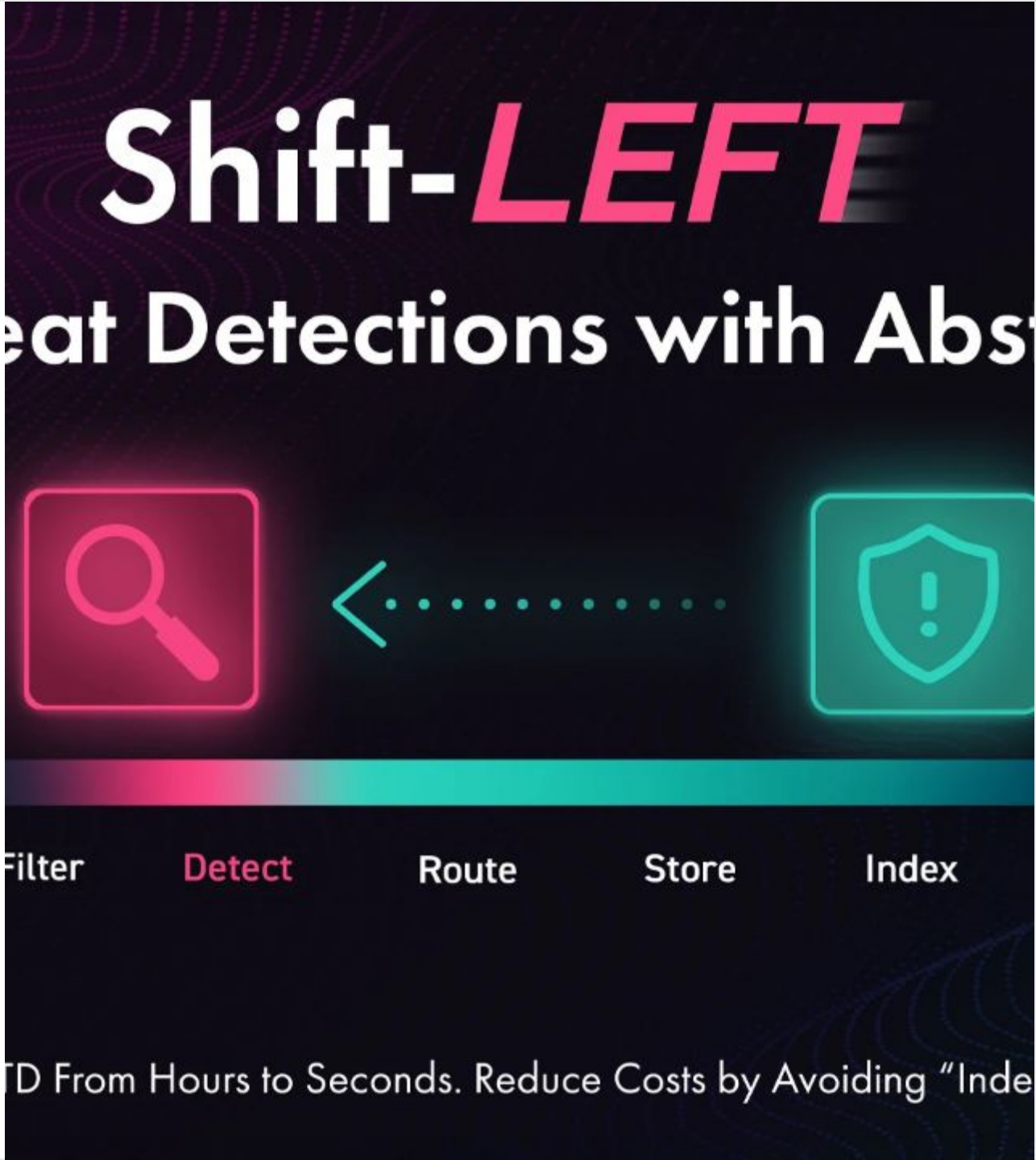
Une approche qui combine agilité et protection continue.

- **Automatisation :** Ne pas ralentir le flux
- **Continuité :** Sécurité à chaque commit
- **Culture :** Responsabilité partagée

LE PRINCIPE "SHIFT LEFT"

Déplacer la sécurité vers la gauche signifie l'intégrer dès les premières étapes du cycle de développement (Plan & Code).

- Détecter les failles au plus tôt
- Faciliter la remédiation par le développeur
- Éviter les blocages en fin de cycle



POURQUOI "SHIFT LEFT" ?

Simplicité

Plus une erreur est détectée tôt, plus elle est techniquement simple à corriger.

Économie

Réduction drastique des ressources mobilisées pour les corrections post-déploiement.

Confiance

Les équipes de production reçoivent des artefacts déjà validés et sécurisés.

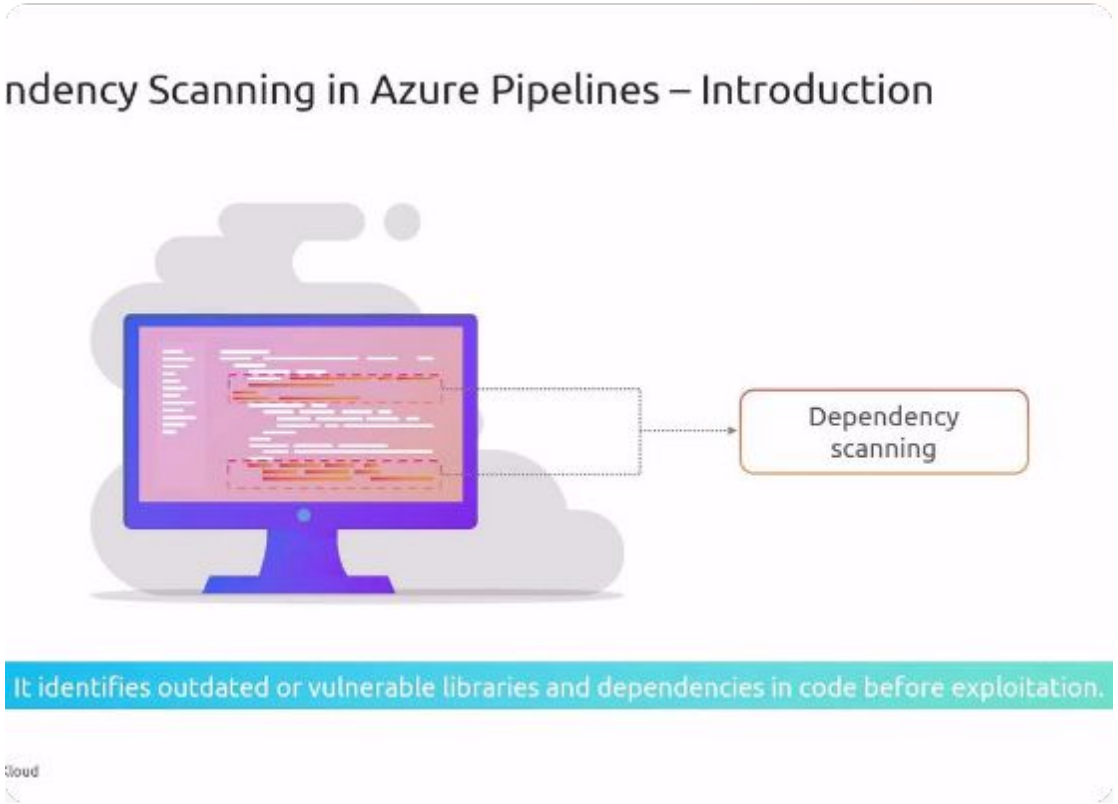
III. Panorama des Risques Modernes

Identifier les menaces dans l'écosystème DevOps

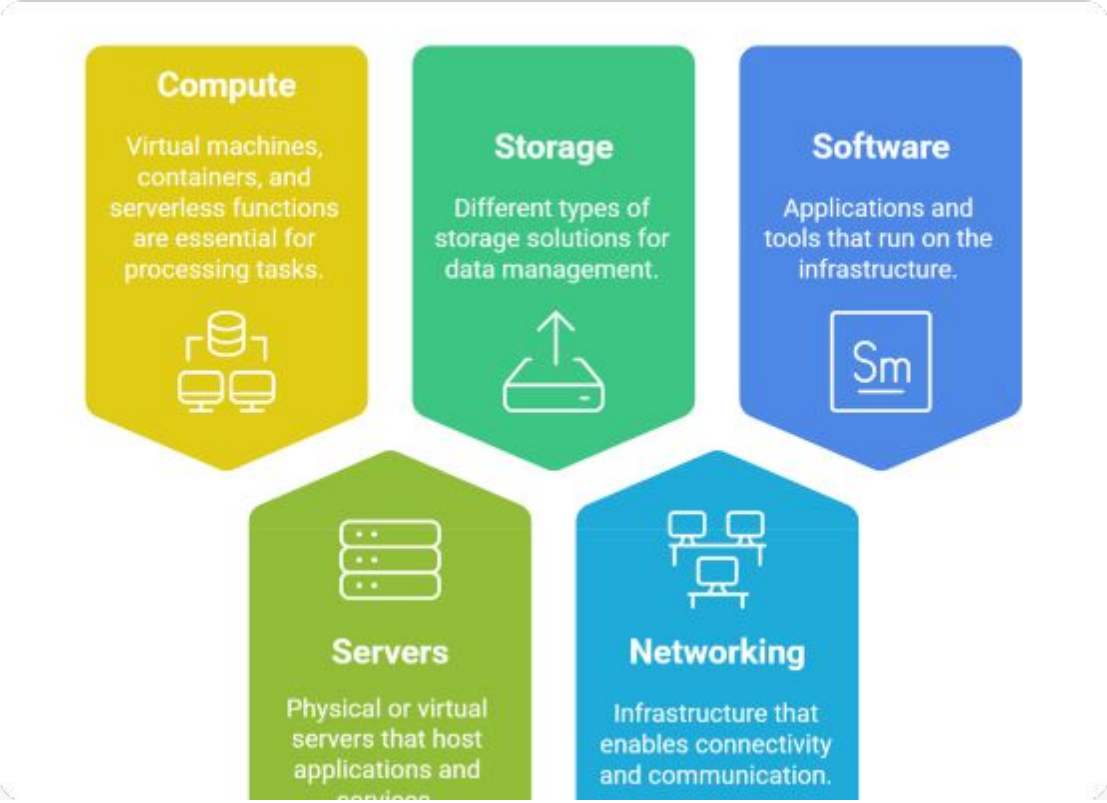
RISQUE 1 : LES DÉPENDANCES

Les applications modernes reposent à 80% sur des bibliothèques Open Source.

- Versions obsolètes contenant des failles connues
- Attaques sur la "Supply Chain"
- Le code peut être vulnérable sans erreur interne



RISQUE 2 : MAUVAISES CONFIGURATIONS



La majorité des incidents Cloud proviennent d'erreurs de configuration.

- Ports ouverts inutilement sur Internet
- Permissions S3/Storage trop larges
- Règles de firewall mal définies ou oubliées

RISQUE 3 : SECRETS EXPOSÉS

L'erreur humaine la plus dangereuse en DevOps : le commit de secrets.

- Mots de passe en clair dans les fichiers
- Clés API commitées accidentellement dans Git
- Tokens de services exposés dans les logs

RISQUE 4 : FAILLES APPLICATIVES

- **Injection SQL :** Manipulation des requêtes DB
- **XSS :** Injection de scripts malveillants
- **Authentification faible :** Sessions mal gérées
- **Défaut de contrôle d'accès :** Insecure Direct Object References (IDOR)

IV. L'Arsenal des Outils de Sécurité

Comprendre SAST et DAST pour une protection complète

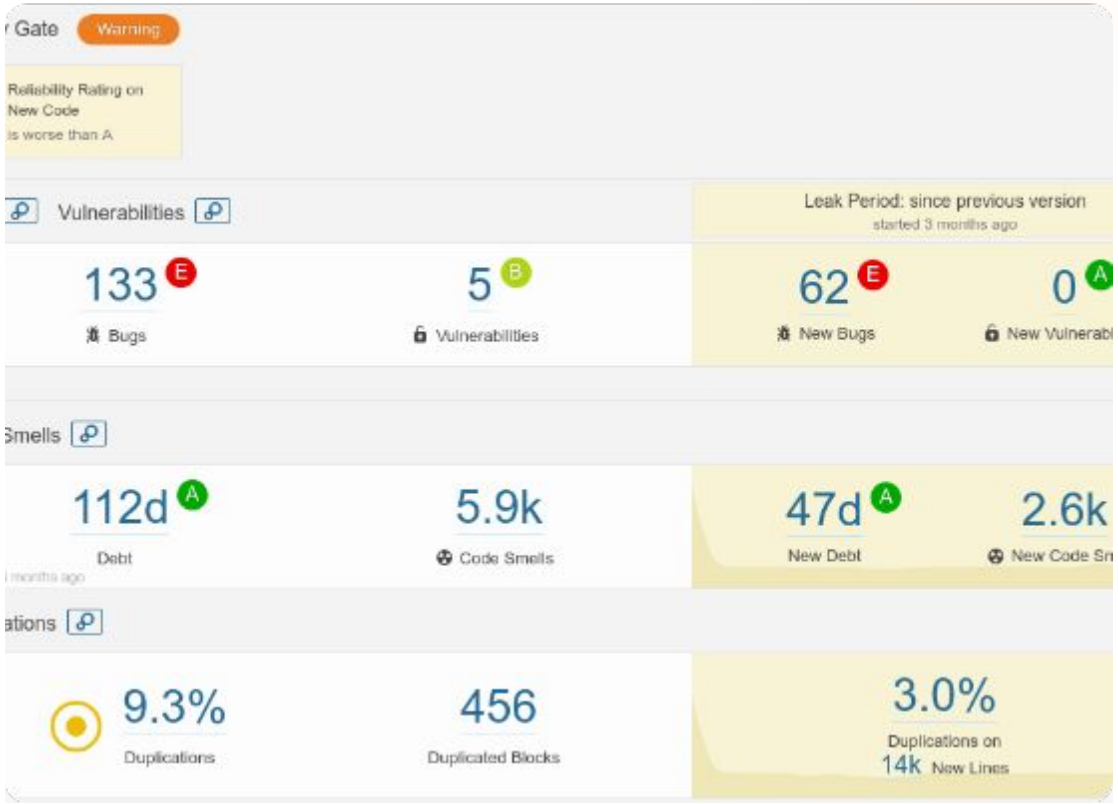
ANALYSE STATIQUE VS DYNAMIQUE

Caractéristique	SAST (Statique)	DAST (Dynamique)
Analyse	Code source (sans exécution)	Application en cours d'exécution
Moment	Phases précoces (Build/Test)	Phases tardives (Staging/Prod)
Détecte	Bugs logiques, secrets, syntaxe	Failles runtime, config serveur
Exemple	SonarQube	OWASP ZAP

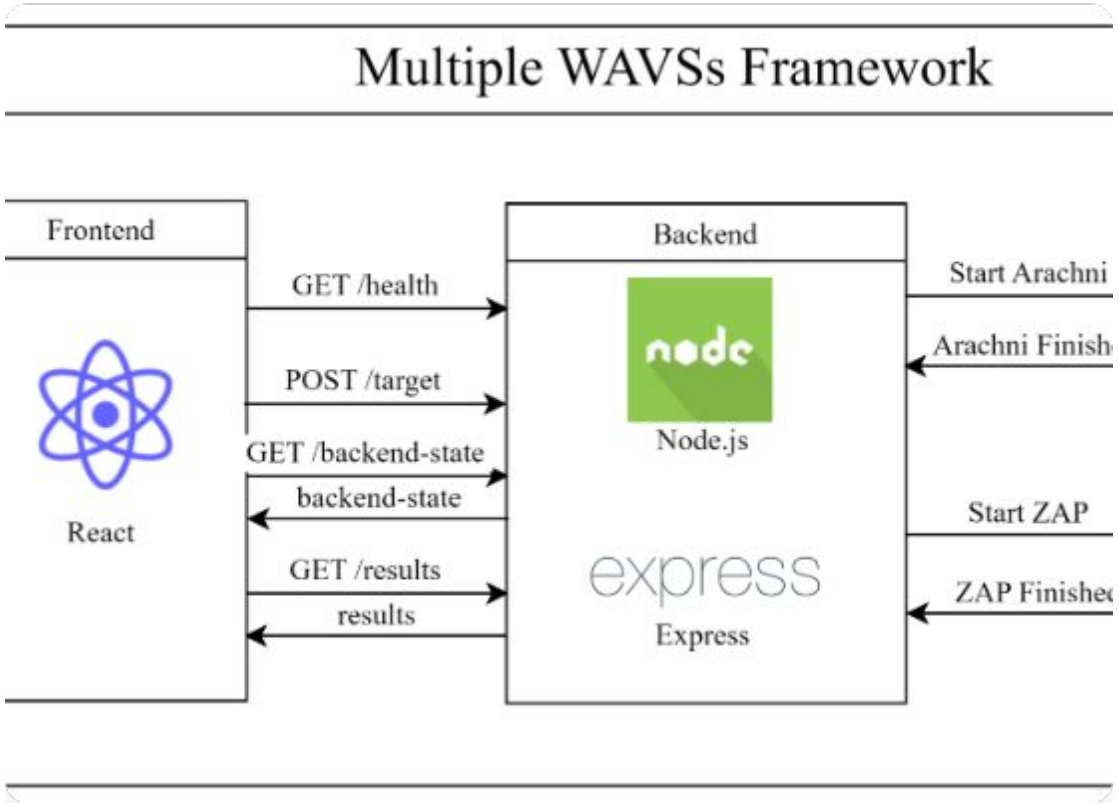
FOCUS SAST : SONARQUBE

L'outil de référence pour la qualité et la sécurité du code.

- Analyse de la dette technique
- Détection des vulnérabilités potentielles
- Mesure de la couverture de tests
- Intégration native dans Jenkins



FOCUS DAST : OWASP ZAP



Simule un attaquant pour tester l'application réelle.

- Scan actif et passif des endpoints
- Détection de failles de configuration HTTP
- Test des formulaires et des sessions
- Génération de rapports de vulnérabilité

V. Automatisation avec Jenkins

Bâtir un pipeline de livraison sécurisé

PIPELINE DEVSECOPS TYPIQUE

Build
Compilation du code source

Unit Tests
Validation fonctionnelle

SAST
SonarQube Scan

Deploy
Staging Environnement

DAST
OWASP ZAP Scan

Release
Prod Déploiement

NOTION DE SEUILS (QUALITY GATES)

0

Vulnerabilités Critiques

Le concept de **Quality Gate** permet de stopper automatiquement le pipeline si les critères de sécurité ne sont pas remplis.

- Évite le "On corrigera plus tard"
- Force la qualité dès le commit
- Garantit un niveau de sécurité minimal

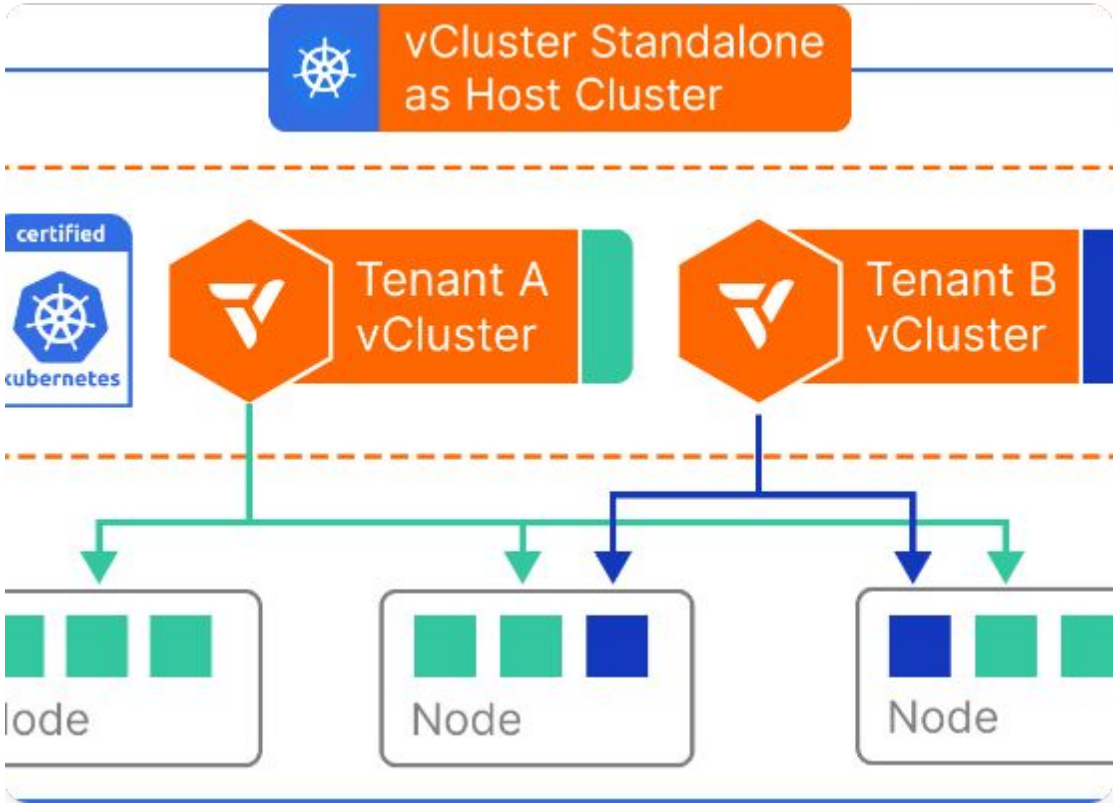
VI. Sécurité Kubernetes (K8s)

Maîtriser l'orchestrateur et ses privilèges

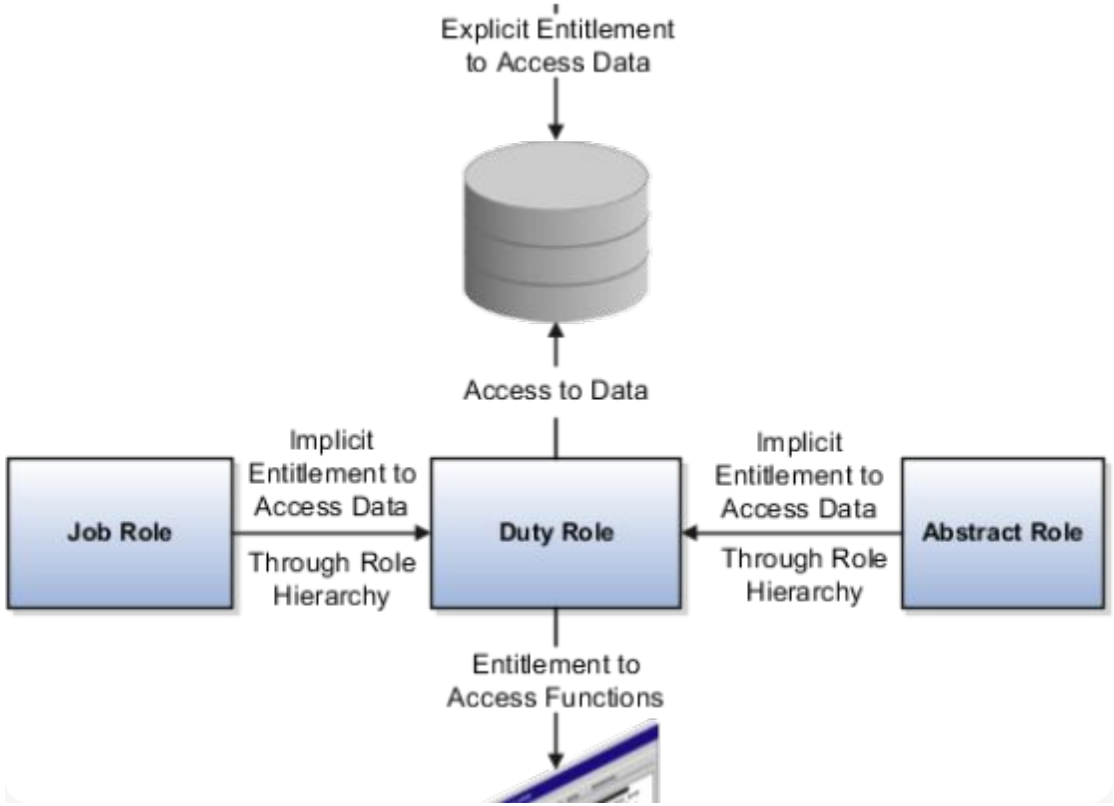
POURQUOI K8S EST SPÉCIFIQUE ?

Kubernetes contrôle l'infrastructure complète.

- Un accès compromis peut paralyser tout le cluster
- Complexité du réseau et des permissions
- Nécessité de protéger les Pods et les Secrets



LE CONTRÔLE D'ACCÈS (RBAC)



Concept	Description
Role	Permissions par Namespace
ClusterRole	Permissions globales
RoleBinding	Associe Rôle → Utilisateur
ServiceAccount	Compte pour les Apps

POLITIQUES ET BONNES PRATIQUES

Moindre Privilège

Donner uniquement les droits strictement nécessaires à chaque service.

Limites CPU/RAM

Empêcher un pod malveillant ou bogué de saturer le cluster complet.

Images Vérifiées

N'autoriser que des images provenant de registres privés et scannés.

Synthèse

- **DevSecOps :** La sécurité est l'affaire de tous
- **Shift-Left :** Détecter tôt = Corriger vite
- **Outils :** SonarQube & ZAP sont vos alliés
- **Kubernetes :** RBAC est indispensable
- **Secrets :** Jamais de secrets dans Git
- **Surveillance :** Monitorer pour réagir

Merci de votre attention. Avez-vous des questions ?